

Outline

Why tune?

Manual tuning

Grid search

Random search

Bayesian optimization

Wrap-up

Parameters vs. hyperparameters

Parameters θ are what the model *learns* from data.

- ▶ Linear regression: the coefficients (one per feature + intercept).
- ▶ Decision tree: the split points and feature choices at each node.
- ▶ Neural net: all the weights and biases.

Hyperparameters λ are what the model *doesn't* learn - you have to choose them.

- ▶ Ridge regression: alpha (regularization strength).
- ▶ Decision tree: max_depth, min_samples_leaf.
- ▶ Neural net: learning rate, number of layers, batch size, weight decay.

Different hyperparameter choices $\Rightarrow$ different fitted models $\Rightarrow$ different generalization.

Tuning = systematically picking λ to optimize a validation/CV score.

Predict-first: how many hyperparameters?

Predict-first: how many hyperparameters?

Predict the count of meaningfully tunable hyperparameters for each model:

- ▶ Ridge regression (sklearn)
- ▶ Random forest (sklearn)
- ▶ XGBoost
- ▶ GPT-style transformer training

Predict-first: how many hyperparameters?

Predict the count of meaningfully tunable hyperparameters for each model:

- ▶ Ridge regression (sklearn)
- ▶ Random forest (sklearn)
- ▶ XGBoost
- ▶ GPT-style transformer training

Rough answers:

- ▶ **Ridge:** 1 (`alpha`). Most people only tune this.
- ▶ **Random forest:** 4-6 (`n_estimators`, `max_depth`, `min_samples_leaf`, `max_features`, `bootstrap`).
- ▶ **XGBoost:** 10-15 sensitive ones (`learning_rate`, `max_depth`, `subsample`, `colsample_bytree`, `min_child_weight`, `regularization`, ...).
- ▶ **Transformer training:** dozens (architecture + optimization + data + regularization).

Tuning effort scales with the search-space size. Method choice should too.

The bilevel optimization view

Tuning is an *outer* optimization wrapping the model's own *inner* optimization.

Outer loop (tuning): pick best λ^*

Inner loop (training): given λ , solve

$$\hat{\theta}(\lambda) = \arg \min_{\theta} \mathcal{R}_{\text{emp}}(\theta; \lambda)$$

pass λ in $\rightarrow$ get back $\hat{\theta}(\lambda)$, score it on validation/CV $\rightarrow$ outer search picks next λ

Choices on outer search strategy:

- ▶ **Manual** - you propose λ values from intuition.
- ▶ **Grid** - exhaustive over a Cartesian product.
- ▶ **Random** - sample uniformly from distributions.
- ▶ **Bayesian** - build a surrogate model of score vs. λ , query smartly.

Manual tuning: “grad student descent”

The simplest strategy: a human proposes one λ , evaluates, then proposes another based on what they saw.

Typical workflow:

1. Start with the library's defaults. Get a baseline.
2. Identify the most-likely-impactful hyperparameter (domain knowledge or quick experiments).
3. Sweep it across ~ 5 values, plot validation score, pick the best.
4. Fix that value, move to the next hyperparameter, repeat.
5. Stop when changes don't improve validation score.

Also called *manual coordinate descent* - you adjust one hyperparameter at a time.

When manual works (and when it doesn't)

Manual is best when:

- ▶ You have strong intuition about the model (you've trained 50 Ridge regressions already).
- ▶ The search space is small (1-2 hyperparameters).
- ▶ Each evaluation is slow (you don't want to launch 100 jobs by accident).
- ▶ You're *developing* a sense for the model, not just optimizing.

Manual breaks down when:

- ▶ More than ~ 3 hyperparameters interact (you can't visualize the surface).
- ▶ You can't afford to be inconsistent across runs.
- ▶ Multiple people work on the same model (reproducibility).
- ▶ You're shipping to production and need a defensible methodology.

Manual tuning teaches you the most per evaluation but scales the worst.

Pitfalls of manual tuning

1. **Confirmation bias.** You notice the run that worked, forget the three that didn't.
Mitigation: write down every run, including failures.
2. **Validation-set abuse.** The 50th hyperparameter setting you tried still gets evaluated on the same validation set. You're effectively overfitting validation.
Mitigation: cross-validation, separate test set, count your evaluations.
3. **No reproducibility.** "I'll try $\alpha=0.3$ next... or was it 0.03?" Mitigation: a notebook log, or skip manual and use a configured automated search.
4. **Local optimum.** Coordinate descent gets stuck. The best Ridge α *given the current data preprocessing* may differ from the best after you change features.
Mitigation: redo the whole sweep occasionally.

Grid search: exhaustive over a Cartesian product

Pick a finite list of candidate values for each hyperparameter. Evaluate *every* combination.

Example for Ridge with polynomial features:

Hyperparameter	Candidate values
alpha	$\{10^{-3}, 10^{-2}, 10^{-1}, 1, 10, 100\}$
poly_degree	$\{1, 2, 3, 4, 5\}$

Total: $6 \times 5 = 30$ configurations. With 5-fold CV, $30 \times 5 = 150$ model fits.

Exhaustive, reproducible, embarrassingly parallel. Also potentially *very* expensive.

Predict-first: combinatorial blowup

You want to tune XGBoost with 4 hyperparameters, 5 candidate values each, using 5-fold CV.

Predict-first: combinatorial blowup

You want to tune XGBoost with 4 hyperparameters, 5 candidate values each, using 5-fold CV.

Predict:

- ▶ How many model fits will grid search require?
- ▶ If each fit takes 30 seconds, how long does the whole search take (single-threaded)?

Predict-first: combinatorial blowup

You want to tune XGBoost with 4 hyperparameters, 5 candidate values each, using 5-fold CV.

Predict:

- ▶ How many model fits will grid search require?
- ▶ If each fit takes 30 seconds, how long does the whole search take (single-threaded)?

Answers:

- ▶ $5^4 \times 5 = 625 \times 5 = \mathbf{3125}$ model fits.
- ▶ $3125 \times 30\text{s} \approx 26$ hours single-threaded.

Add one more hyperparameter (5 values): **15,625** fits, **5.4** days. The curse of dimensionality strikes early.

Code: GridSearchCV with Pipeline

```
from sklearn.model_selection import GridSearchCV
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.linear_model import Ridge
from sklearn.pipeline import Pipeline

pipe = Pipeline([("poly", PolynomialFeatures()),
                  ("scale", StandardScaler()),
                  ("ridge", Ridge())])

grid = {
    "poly__degree": [1, 2, 3, 4, 5],
    "ridge__alpha": [1e-3, 1e-2, 1e-1, 1, 10, 100],
}

gs = GridSearchCV(pipe, grid, cv=5,
                  scoring="neg_mean_squared_error", n_jobs=-1)
gs.fit(X_train, y_train)
print(gs.best_params_, -gs.best_score_)
```

`n_jobs=-1` uses all CPU cores. Parameters are accessed as `step_name__param_name`.

When grid is OK

- ▶ $\leq 2-3$ hyperparameters AND each has ≤ 6 candidate values.
- ▶ Each model fit is cheap (small data, simple model).
- ▶ Reproducibility / publication: “we evaluated all X configurations.”
- ▶ Inputs have clearly preferred discrete values (e.g., `max_depth` $\in \{3, 5, 7\}$).

Once you have 4+ hyperparameters or 10+ values each, grid stops being practical.
Random search is almost always better.

Why grid wastes compute in high dimensions

Most hyperparameters don't matter equally. For a typical Ridge + polynomial model:

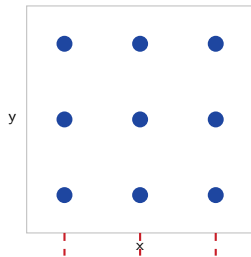
- ▶ `alpha` matters a lot.
- ▶ `poly_degree` matters a lot.
- ▶ Other small dials (e.g., `tol`, `max_iter`) barely affect the score.

Grid search spends *equal* compute on each axis. If 9 out of 10 axes are flat, you've used 90% of your budget exploring flat directions.

Random search samples each hyperparameter from a distribution. Every trial visits a new value on *every* axis - including the ones that matter.

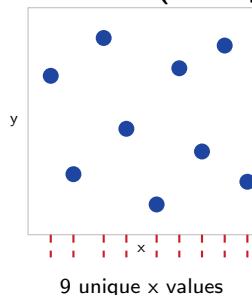
Bergstra–Bengio: random $>$ grid in high dimensions

Classic visualization. 9 trials in 2D. Imagine the *horizontal* axis matters, the *vertical* axis doesn't.



Grid search (3x3)

Random search (9 samples)



Random search gets 9 distinct samples along the important axis; grid gets 3. The wider the gap between effective dimensionality and total dimensionality, the bigger the random-search win.

Random search advantage in one line

If only d_{eff} of your d hyperparameters matter (and you don't know which), random search with n trials finds a near-optimal value on each important axis - because every trial is a fresh sample on every axis.

Rule of thumb: for $d \geq 4$ hyperparameters, random search with 60 trials typically matches or beats grid search with 10^d trials.

Bonus benefits:

- ▶ You can stop anytime (early-stopping the search). Grid forces you to finish.
- ▶ Easy to bound budget: set `n_iter`.
- ▶ Mixes naturally with continuous distributions (`loguniform`, `uniform`, `normal`).

Code: RandomizedSearchCV

```
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import loguniform, randint

dist = {
    "ridge__alpha": loguniform(1e-4, 1e3),    # continuous, log-
        scale
    "poly__degree": randint(1, 7),            # discrete 1..6
}

rs = RandomizedSearchCV(pipe, dist,
    n_iter=60,                                # budget
    cv=5,
    scoring="neg_mean_squared_error",
    random_state=509,                          # reproducibility
    n_jobs=-1)
rs.fit(X_train, y_train)
print(rs.best_params_, -rs.best_score_)
```

Use `loguniform` for hyperparameters that span orders of magnitude (regularization strength, learning rate). Use `uniform` for bounded ranges. Use `randint` for integer-valued discrete ones.

Bayesian optimization: learn from each evaluation

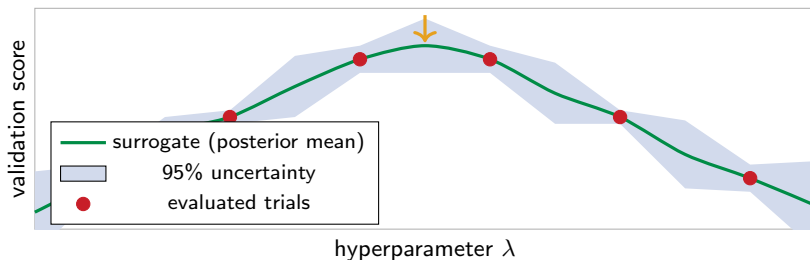
Random search treats each trial as independent. But you've *seen* the previous scores - you could pick the next trial based on what looks promising.

Bayesian optimization (BO):

1. Fit a probabilistic **surrogate model** to the (hyperparameter $\rightarrow$ score) pairs seen so far. Typically a Gaussian Process or Tree-structured Parzen Estimator (TPE).
2. Use the surrogate's mean and uncertainty to compute an **acquisition function** that scores "how good would it be to evaluate here?"
3. Pick the next hyperparameter as the maximum of the acquisition function.
4. Evaluate, add to history, update surrogate. Repeat.

Each trial uses the information from all previous trials. Random search doesn't.

The surrogate + acquisition picture



The surrogate is confident near observed points (narrow band) and uncertain elsewhere (wide band). The acquisition function trades off *exploit* (go where the mean is high) and *explore* (go where the uncertainty is high).

Acquisition functions

Three popular acquisition functions:

- ▶ **Expected Improvement (EI)**. How much better do we expect to do here than at the best-seen point? Smooth tradeoff between mean and uncertainty. Default in most libraries.
- ▶ **Upper Confidence Bound (UCB)**. $\text{mean} + \kappa \cdot \text{std}$. The constant κ explicitly controls explore-vs-exploit. Used in multi-armed-bandit theory.
- ▶ **Probability of Improvement (PI)**. Probability that the score here exceeds the current best. Greedy - tends to under-explore.

The acquisition function is itself optimized over λ (cheap - no model fits required), then we evaluate the model only at the chosen point.

Code: Optuna (modern default)

```
import optuna
from sklearn.model_selection import cross_val_score

def objective(trial):
    alpha = trial.suggest_float("alpha", 1e-4, 1e3, log=True)
    degree = trial.suggest_int("degree", 1, 6)

    pipe = make_pipeline(PolynomialFeatures(degree),
                          StandardScaler(),
                          Ridge(alpha=alpha))
    score = cross_val_score(pipe, X_train, y_train,
                             cv=5, scoring="neg_mean_squared_error").
              mean()

    return score                                # Optuna maximizes by default

study = optuna.create_study(direction="maximize")
study.optimize(objective, n_trials=60)
print(study.best_params, study.best_value)
```

Default sampler: TPE (Tree-structured Parzen Estimator). Free pruning + dashboard.

When Bayesian opt is worth it

Use Bayesian opt when:

- ▶ Each evaluation is expensive (training takes minutes-hours, not seconds).
- ▶ You have ≥ 4 hyperparameters with non-trivial interactions.
- ▶ You want fewer total evaluations than random search would need.
- ▶ You can keep evaluations sequential (Bayesian opt is inherently sequential, though batch variants exist).

Stick with random search when:

- ▶ Evaluations are cheap (seconds) - the overhead of fitting a surrogate isn't justified.
- ▶ You can run hundreds of parallel evaluations on a cluster - random is embarrassingly parallel; Bayesian is mostly sequential.
- ▶ Your search space is largely categorical with no obvious order.

Modern AutoML tools (Optuna, Hyperopt, Ray Tune) default to TPE-style Bayesian samplers because they “just work” across mixed search spaces.

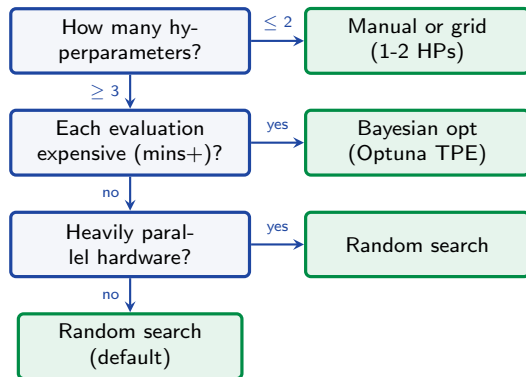
The compute budget reality

Hyperparameter tuning consumes most of an ML project's compute. Practical patterns:

- ▶ **Subsample.** Tune on 10% of training data, retrain best config on full data. Often within 1% of fully-tuned.
- ▶ **Early stopping.** For boosted trees and NNs, stop training early when validation stops improving. Cuts each trial's cost by 2-5x.
- ▶ **Pruning.** Stop trials that look bad after a few epochs (Optuna does this for you).
- ▶ **Multi-fidelity methods.** Hyperband / Successive Halving: run many trials cheaply, gradually allocate more budget to the survivors.
- ▶ **Stop when score plateaus.** Track best-so-far; if it hasn't improved in 20 trials, you're done.

Honest reporting: the score you tuned to optimize is *optimistically biased*. Report the test-set score after tuning is complete (or use nested CV - L05b).

Decision flowchart



Defaults: ≤ 2 HPs $\rightarrow$ grid is fine; otherwise random search; if compute is the bottleneck, Bayesian opt.

Recap

- ▶ **Parameters** are learned from data. **Hyperparameters** are chosen by you. Tuning = outer optimization wrapping the inner fit.
- ▶ **Manual.** Cheap, intuition-building, doesn't scale. Watch for confirmation bias and validation-set abuse.
- ▶ **Grid.** Exhaustive over Cartesian product. Fine for ≤ 2 HPs; explodes in high dimensions.
- ▶ **Random.** Default for ≥ 3 HPs. Each trial is a fresh sample on every axis. Easy to budget and parallelize. Beats grid when effective dimensionality is low.
- ▶ **Bayesian.** Models the score-vs-HP surface and queries smartly. Best when each evaluation is expensive. Optuna's TPE is the modern default.
- ▶ **Honest reporting.** Tuning inflates the score; report on a held-out test set (or nested CV - L05b).

HW01e - practice

Estimated time: 90-120 minutes. Tasks 1-3 graded; bonuses optional.

1. On the Yerevan rent dataset: manually tune Ridge α on $\{0.001, 0.01, 0.1, 1, 10, 100, 1000\}$ using 5-fold CV. Plot validation MSE vs. α on a log-x axis. Report the best.
2. Use `GridSearchCV` to tune $\text{poly_degree} \in \{1, 2, 3, 4, 5\}$ and α (same grid). Report best params and wall-clock time.
3. Replace step 2 with `RandomizedSearchCV`: $\text{poly_degree} \sim \text{randint}(1, 6)$, $\alpha \sim \text{loguniform}(1e-4, 1e3)$, $n_iter=60$. Compare best score and wall-clock time to grid.
4. *Bonus 1*: run Optuna with TPE for 60 trials on the same search space. Compare to random search. Plot Optuna's history.
5. *Bonus 2*: extend to XGBoost with 5 hyperparameters. Run grid (cap at 10 minutes), random ($n_iter = 60$), and Optuna ($n_trials = 60$). Which wins on a fixed time budget?
Most instructive question.
6. *Bonus 3*: wrap your best pipeline in nested CV (L05b) and report an honest generalization score. How much did tuning inflate the optimistic estimate?